# Deep Learning

## VAEs, Diffusion Models

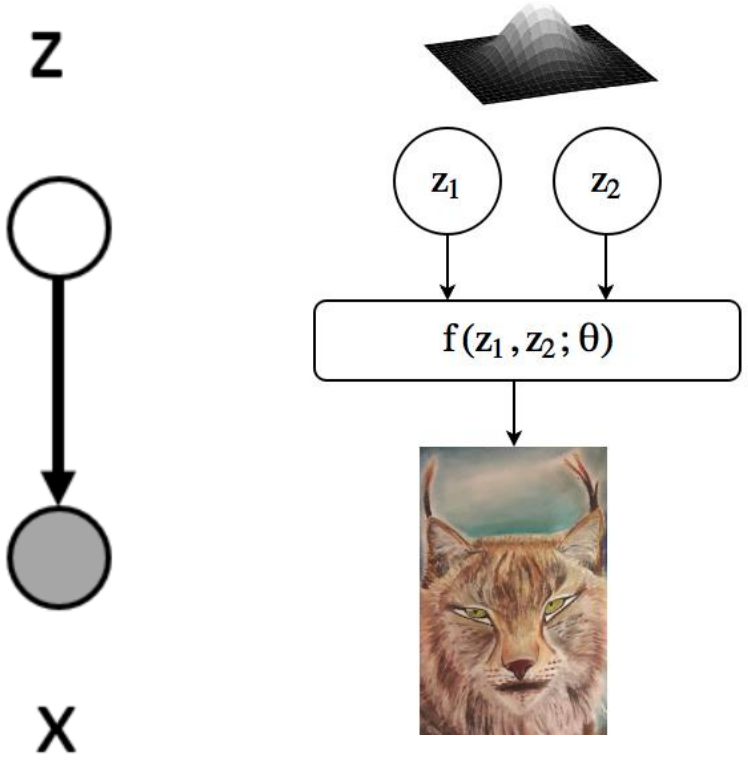
Denis Derkach, Dmitry Tarasov

Laboratory for methods of big data analysis

Spring 2026

# Latent Variable Models

# Latent Variable Models: Motivation




- **Observe** $X$ in the data sample. Latent observables $Z$ are hidden.
- Knowing latent variable space correctly can be beneficial as $p(x|z)$ can be described **simpler** than $p(x)$.
- **Hard** to find the real $Z$ space manually, we need unsupervised learning.

# Gaussian Mixture Model

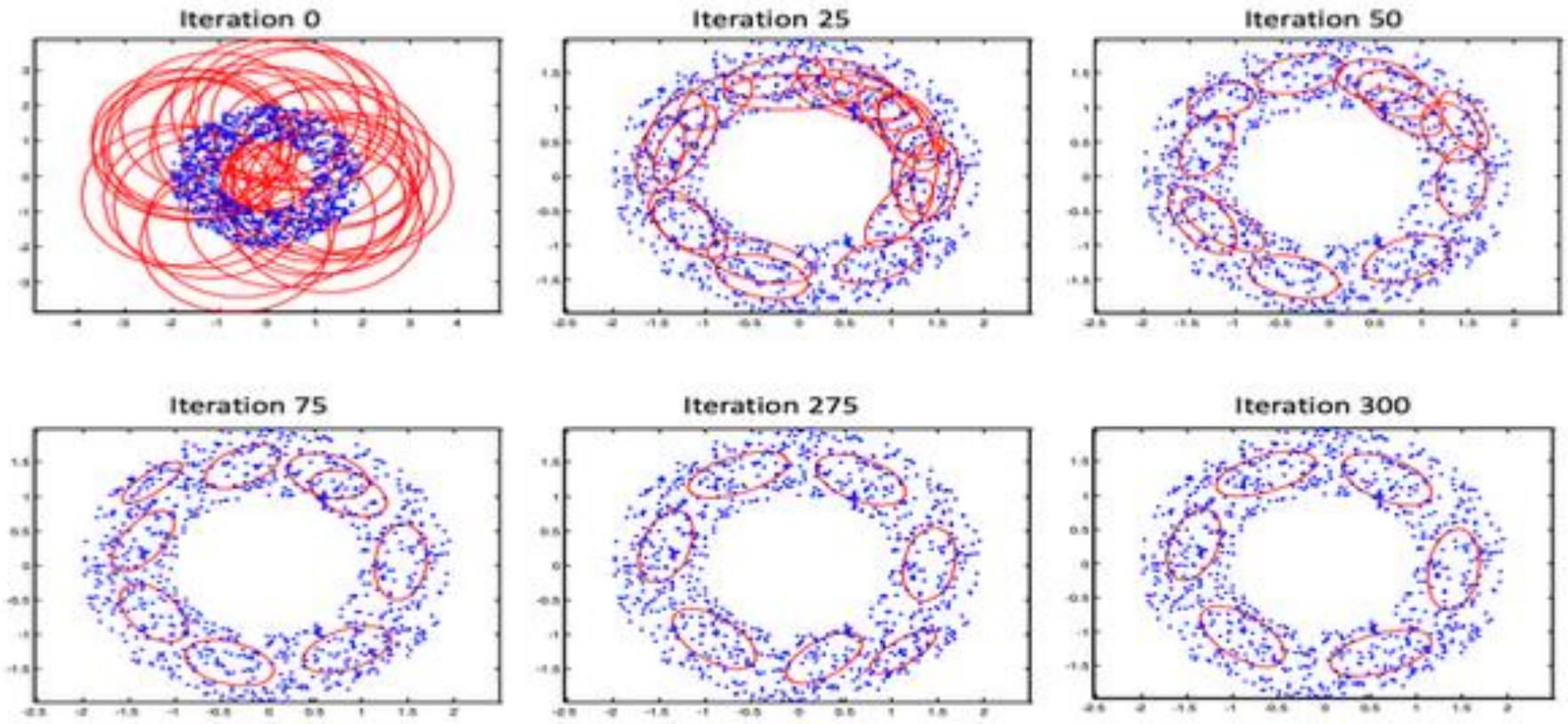
We can use a mixture of distributions (like, Gaussians) to have a better estimate of true PDF. Gaussian Mixture Model proposes:

$$p_{GMM} = \sum_{l=1}^{K} \pi_l \phi(x; \mu_l, \sigma_l^2),$$

where $\pi_l \geq 0$ are the weights: $\sum \pi_l = 1$. Here the number K is a tuning parameter that specifies the number of Gaussians in our model.

# Gaussian Mixture Model

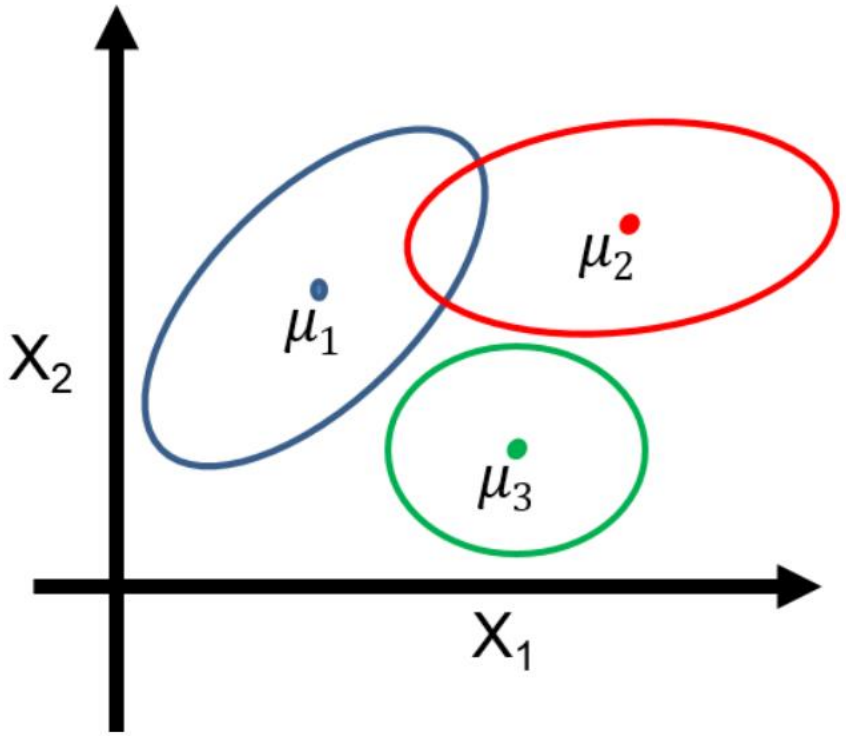
Training set: $n = 900$ examples from a uniform pdf inside an annulus, model: GMM with $K = 30$ Gaussian components



# Mixture of Gaussians as Latent Variable Model

- GMM is a latent variable model.
  - $z \sim \text{Categorical}(1, K)$
  - $p(x|z = k) = N(\mu_k, \Sigma_k)$

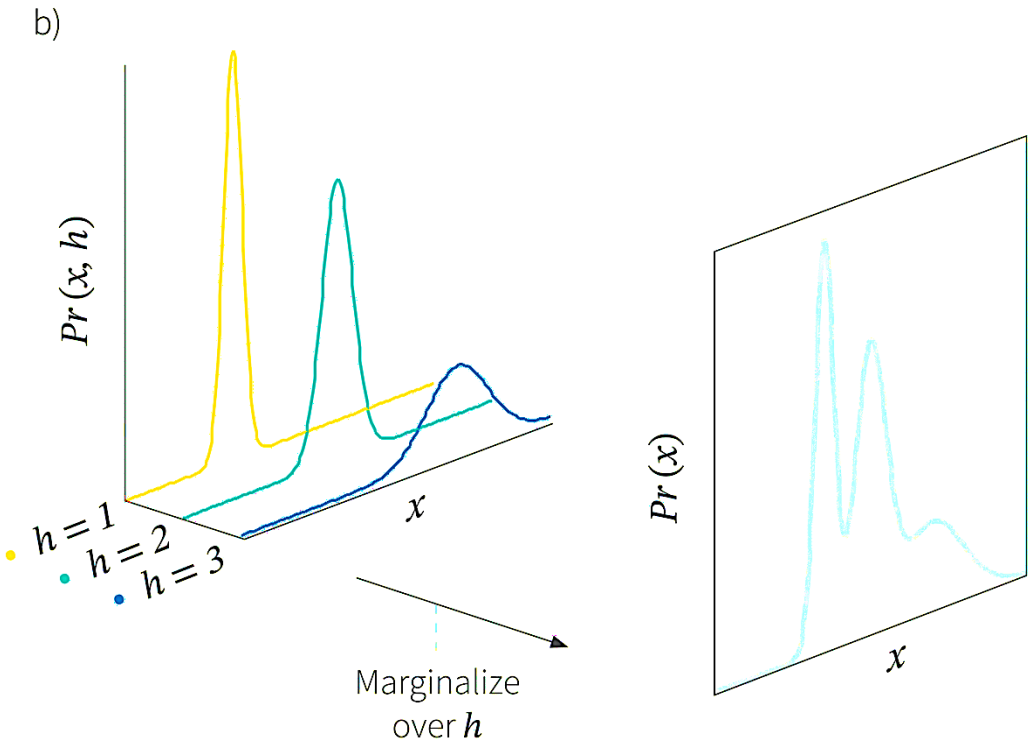
- Generative process
  - Pick a mixture component $k$ by sampling $z$.
  - Generate a data point by sampling from that Gaussian.



# Combination of $p(x)$

- Combine simple models into a more complex and expressive one.

- Obtain PDF using simple formula.

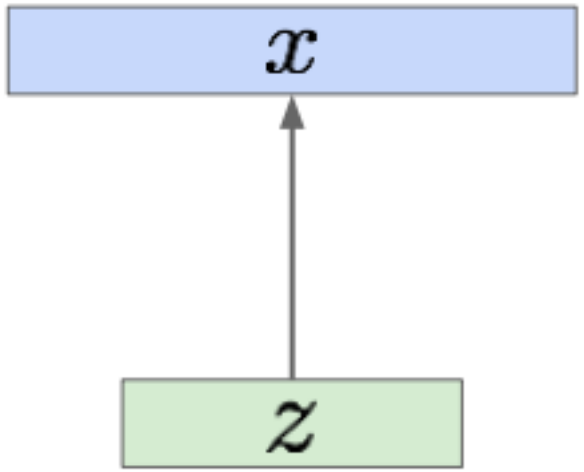



$$p(x) = \sum_{all\ z} p(x, z) = \sum_{all\ z} p(z)p(x|z) = \sum_{k=1}^{K} p(z = k)N(x; \mu_k, \Sigma_k)$$

# Continuous z

- Latent variable model.
  - $\boldsymbol{z} \sim \boldsymbol{p}(\boldsymbol{z})$
  - $p(x|\boldsymbol{z} = \boldsymbol{z}^{(\boldsymbol{i})}) = N(\mu_{(i)}, \Sigma_{(i)})$

- Generative process
  - Pick $z$ from prior.
  - Generate a data point by sampling from Gaussian.

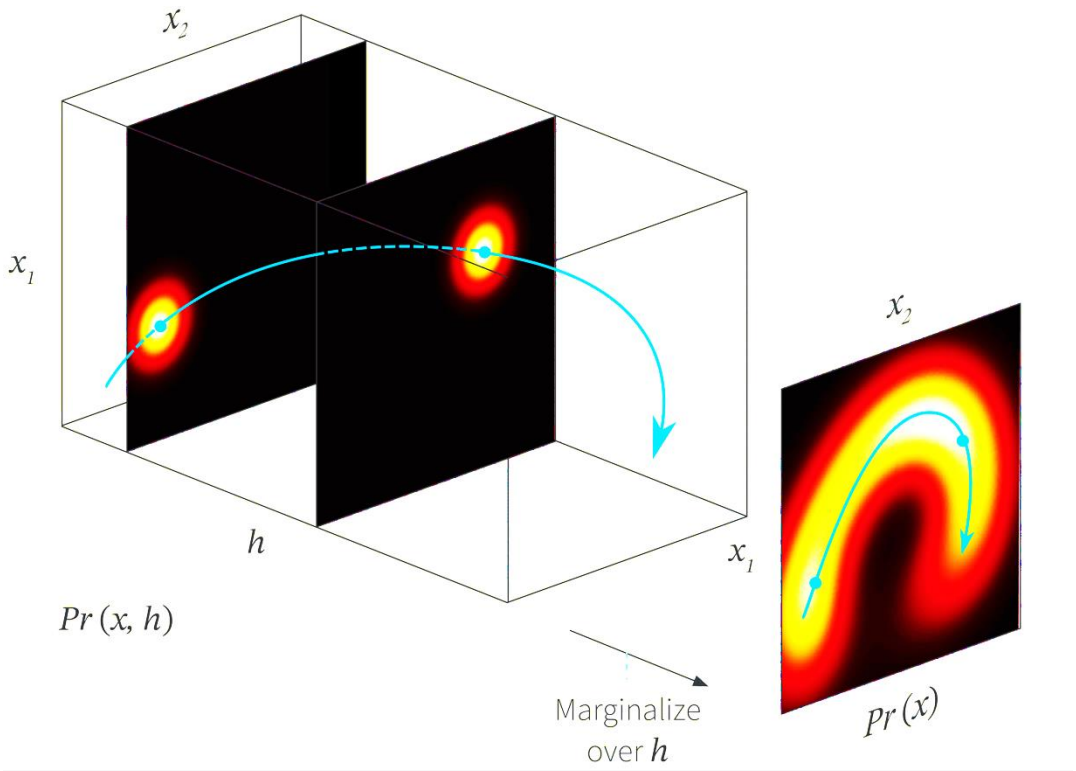


# Combination of $p(x)$

- Combine simple models into a more complex and expressive one.

- Obtain PDF using simple formula.

- What if we have a continuous observable?




$$p(x) = \sum_{all\ z} p(x,z) = \sum_{all\ z} p(z)p(x|z) = \sum_{k=1}^{K} p(z=k)N(x;\mu_k,\Sigma_k)$$

$$p(x;\theta) = \int_z p(x,z;\theta)dz = \int_z p(x;\theta|z)p(z)dz$$

# Finding parameters $\theta$

How do we find the parameters $\theta$? The most natural way is to maximise the log likelihood:

$$\log \prod_{i=1}^{D} p(x^i; \theta) = \sum_{x \in D} \log p(x; \theta)$$

In the following we will need to:

- define a loss function to optimize
- calculate gradients of the loss function with respect to the parameters $\theta$.
- apply a variant of stochastic gradient descent.

# Maximizing the likelihood

Consider one sample, $x$. Then to maximize the likelihood, we need to calculate:

$$p(x;\theta) = \int_z p(x,z;\theta)dz = \int_z p(x;\theta|z)p(z)dz$$

- Easy to calculate in special cases:
  - in case of independent observables $x|z \perp z$ .
  - special cases of conjugation.
- In general, x depends heavily on z (by construction).
- Integral is intractable.

# Wrap-up

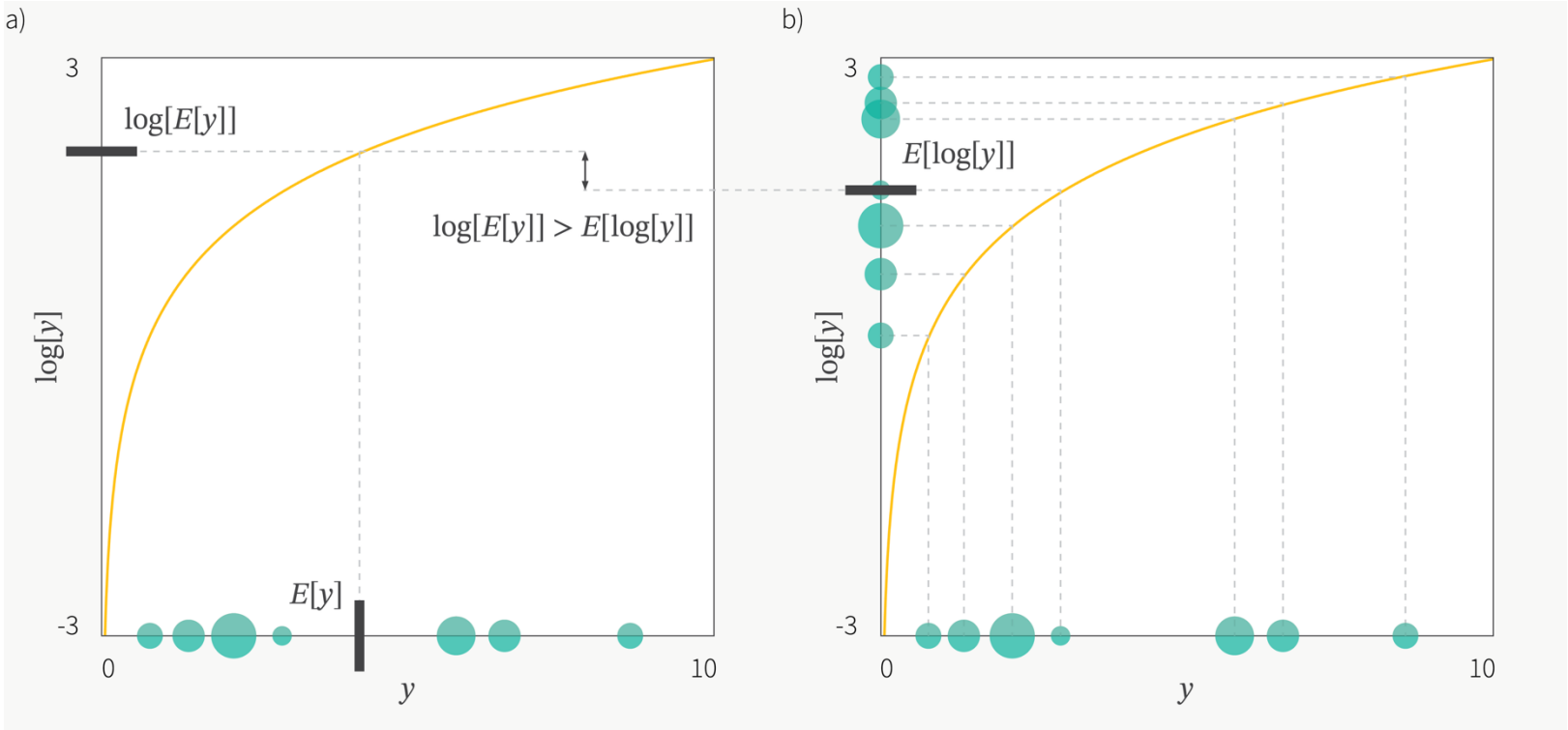
- Problems:
  - We can't calculate p(x).
  - We can't write the maximum-likelihood objective.
- We need a different approach or objective function. Idea:
  - if we can't write likelihood, let's instead derive a lower bound on it.
  - If the lower bound is tractable, then we can optimize the parameters with respect to the lower bound.
  - If we are making the lower bound larger, we are making the likelihood larger.

# Evidence Lower Bound

# Jensen's inequality

For concave function g(.):

$$g[\mathbb{E}[y]] \geq \mathbb{E}[g[y]].$$




$$\log \int p(y) y dy \geq \int p(y) \log y \, dy \; .$$

Figure: BorelisAI

# Deriving the Evidence Lower BOund

- Start from arbitrary function multiplying and dividing $q(z)$:

$$\log p(x;\theta) = \log \int p(x,z;\theta)dz \ = \log \int q(z)\frac{p(x,z;\theta)}{q(z)}dz\,.$$

- Jensen's inequality for convex functions (log):

$$\log \int q(z)\frac{p(x,z;\theta)}{q(z)}dz \geq \int \boldsymbol{q(z)\log\frac{p(x,z;\theta)}{q(z)}dz}\,.$$

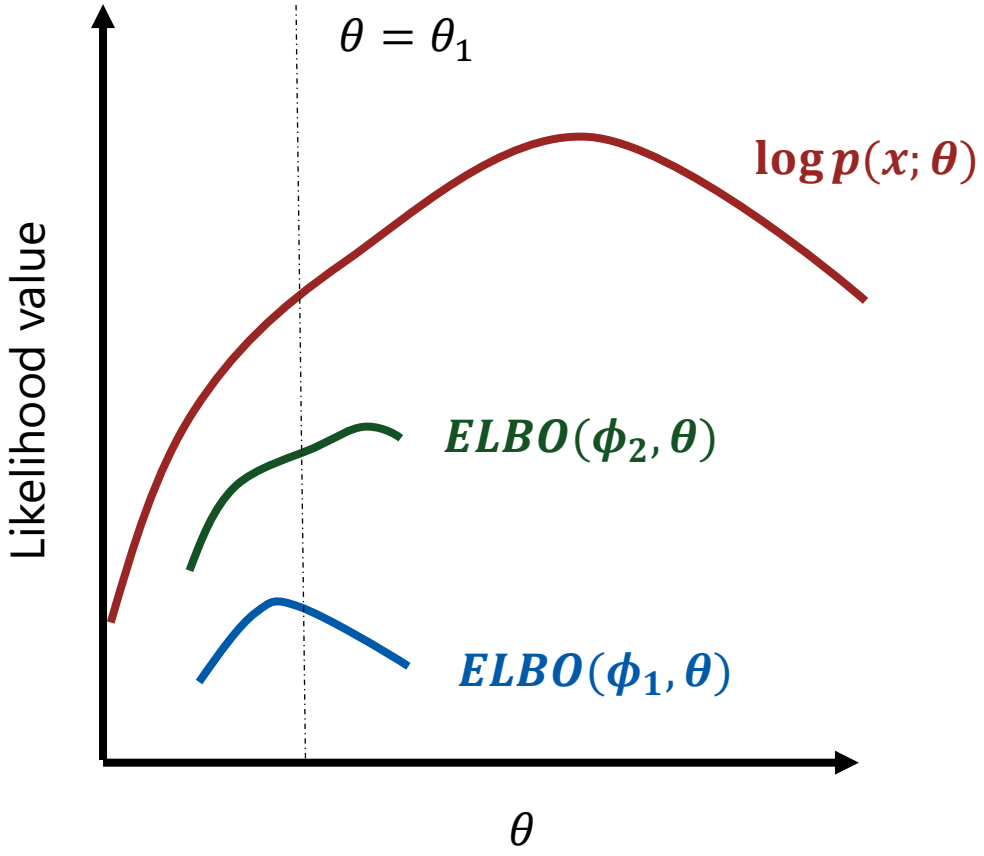
The rhs is **evidence lower bound**

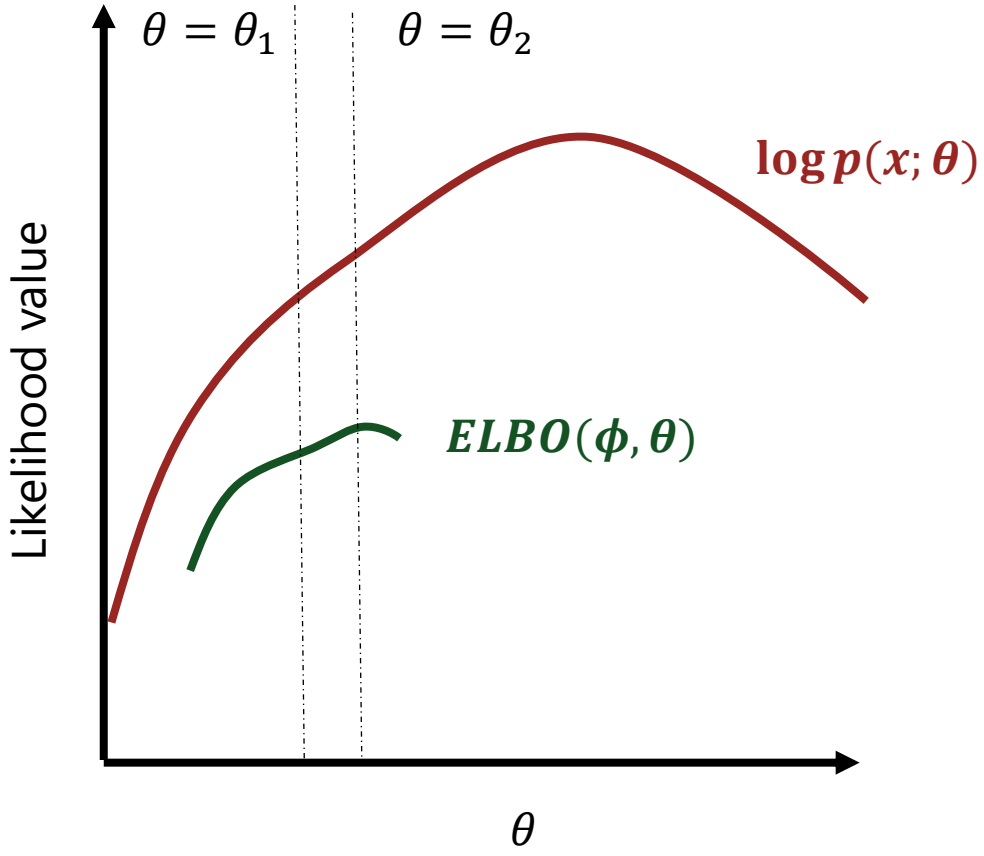
- $q(z)$ has different set of parameters, $\phi$

$$ELBO(\phi,\theta) = \int q(z;\phi)\log\frac{p(x,z;\theta)}{q(z;\phi)}dz$$

# ELBO so far

$$ELBO(\phi, \theta) = \int q(z;\phi) \log \frac{p(x,z;\theta)}{q(z;\phi)} dz$$



# Tightness of bound

$$ELBO(\phi,\theta) = \int q(z;\phi)\log\frac{p(x,z;\theta)}{q(z;\phi)}dz$$

- We use chain rule

$$ELBO(\phi,\theta) = \int q(z;\phi)\log\frac{p(x;\theta)p(z|x;\theta)}{q(z;\phi)}dz$$

- Separate part independent on $z$:

$$= \int q(z;\phi)\log p(x;\theta)\,dz + \int q(z;\phi)\log\frac{p(z|x;\theta)}{q(z;\phi)}dz =$$

$$= \log p(x;\theta) + \int q(z;\phi)\log\frac{p(z|x;\theta)}{q(z;\phi)}dz =$$

- Definition of KL divergence:

$$= \log p(x;\theta) - KL(q(z;\phi)||p(z|x;\theta)).$$

- Bound is tight iff KL = 0.

M. Hoffman, M. Johnson, ELBO surgery, NIPS2016

# Variational Autoencoder

# Dependence on prior on $z$

- We use chain rule

$$ELBO(\phi,\theta) = \int q(z;\phi)\log\frac{p(z)p(x|z;\theta)}{q(z;\phi)}dz \; =$$

- Separate KL divergence part:

$$= \int q(z;\phi)\log p(x|z;\theta)\,dz + \int q(z;\phi)\log\frac{p(z)}{q(z;\phi)}dz =$$

- Definition of KL

$$= \int q(z;\phi)\log p(x|z;\theta)\,dz - KL(q(z;\phi)||p(z)).$$

**Reconstruction loss** **Latent Parametrization agreement**

# Latent Parametrization Agreement

- Measures the degree to which the auxiliary distribution, $q(h, \theta)$, matches the prior $p(z)$:

$$KL(q(z;\phi)||p(z)).$$

- The KL divergence has closed from if both are distributions are known parametric (e.g., Gaussian):

$$q(z;\phi) = N_z(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$
$$p(z) = N_z(\mathbf{0}, \boldsymbol{I}).$$

- In this case:

$$KL[q(z|x,\theta), \Pr(z)] = 12(Tr[\Sigma] + \mu^T\mu - D - \log[\det[\Sigma]])$$

D. Kingma et al. Auto-Encoding Variational Bayes, ICLR 2014

# Latent Parametrization Agreement: training

- Measures the degree to which the auxiliary distribution, $q(h, \theta)$, matches the prior $p(z)$:

$$KL(q(z;\phi)||p(z)).$$

- The KL divergence has closed from if both are distributions are known parametric (e.g., Gaussian):

$$\boldsymbol{q(z|x;\phi) = N_z\left(g_\mu(x;\phi), g_\sigma(x;\phi)\right);}$$

$$p(z) = N_z(\mathbf{0}, \boldsymbol{I}).$$

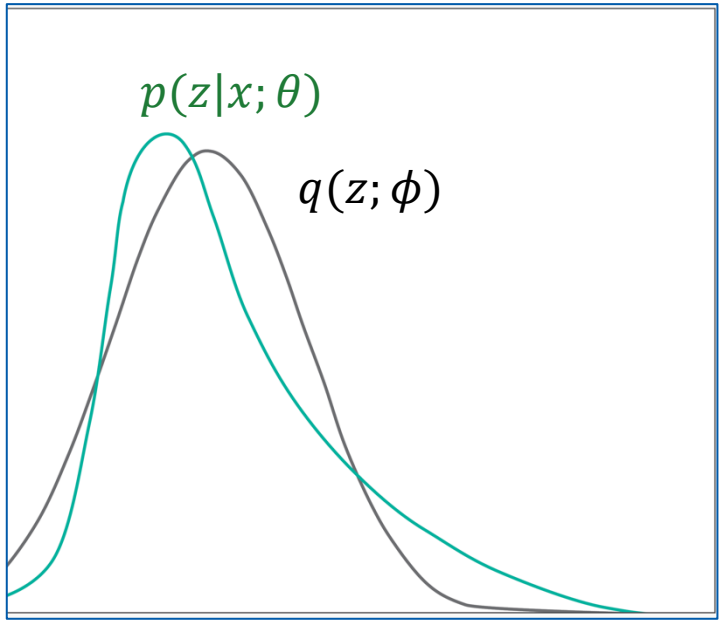
- In this case:

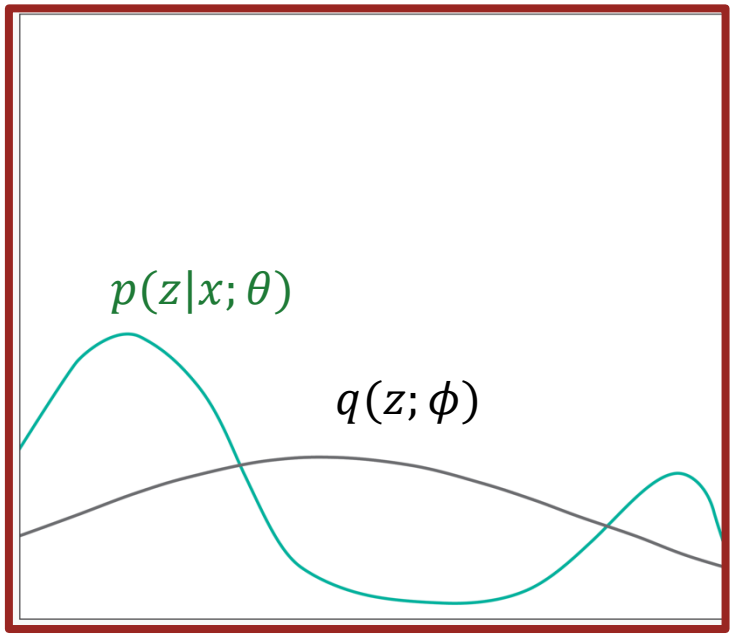
$$KL[q(z|x,\theta), \Pr(z)] = 12(Tr[\Sigma] + \mu^T\mu - D - \log[\det[\Sigma]])$$

# Problems of parametrization choice

- For tightness of bound:

$$KL(q(z;\phi)||p(z|x;\theta)) \to 0.$$




- The tight bound might be unreachable.

Figure: BorelisAI

# Reconstruction Loss

- Reconstruction loss:

$$\int q(z|x;\phi)\log p(x|z;\theta)\,dz\,.$$

- Still troubles to compute directly, we can approximate it with mathematical expectation (even with one sample $z^*$) and thus have:

$$ELBO(\theta,\phi) = \int q(z;\phi)\log p(x|z;\theta)\,dz - KL(q(z;\phi)||p(z)) \approx$$

$$\approx \log[p(x|z^*;\phi)] - KL[q(z|x,\theta),p(z)].$$

# Variational Autoencoder

Maximize
$\mathcal{L}_{VAE} = \mathbb{E}_z \log[p(x|z;\theta)] - KL[q(z|x,\phi), p(z)]$

**Autoencoder**, as we reconstruct $x \to z \to x$.
**Variational**, as it constructs Gaussian approximation to the posterior distribution.

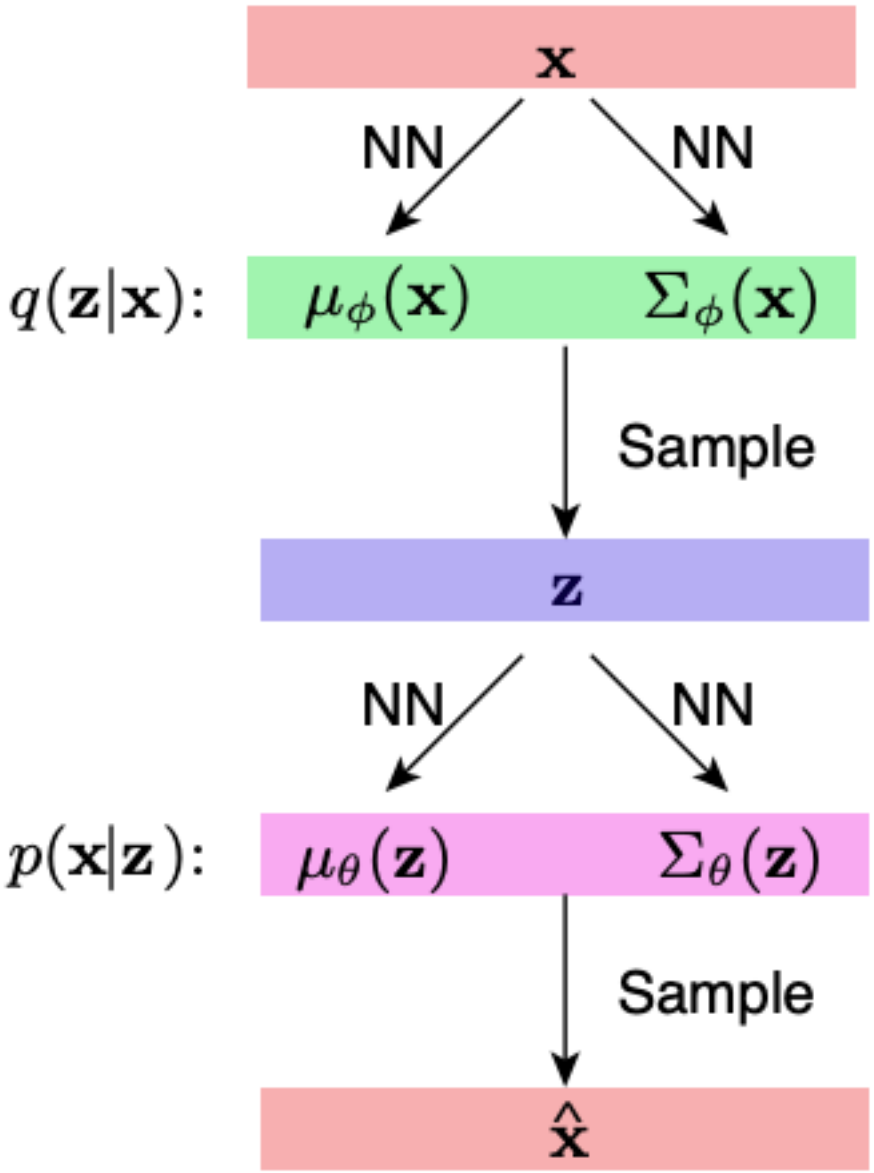



Fig: Kao UCLA DL lectures

# Training VAE

We can estimate gradients for ELBO:

- gradient wrt $\theta$ is easy:

$$\nabla_\theta \mathcal{L}(x; \theta, \phi) = \nabla_\theta \log p(x, z; \theta)$$

  and can be done using a simple MC estimation.

- for $\phi$ situation is much more tricky, since we calculate expectation over $q$:

$$\nabla_\phi \mathcal{L}(x; \theta, \phi) = \nabla_\phi \mathbb{E}_{q(z|x;\phi)}[\log p(x, z; \theta) - \log q(z|x; \phi)].$$

  luckily, we can use reparameterization.

# Reparameterization trick

- Sampling:

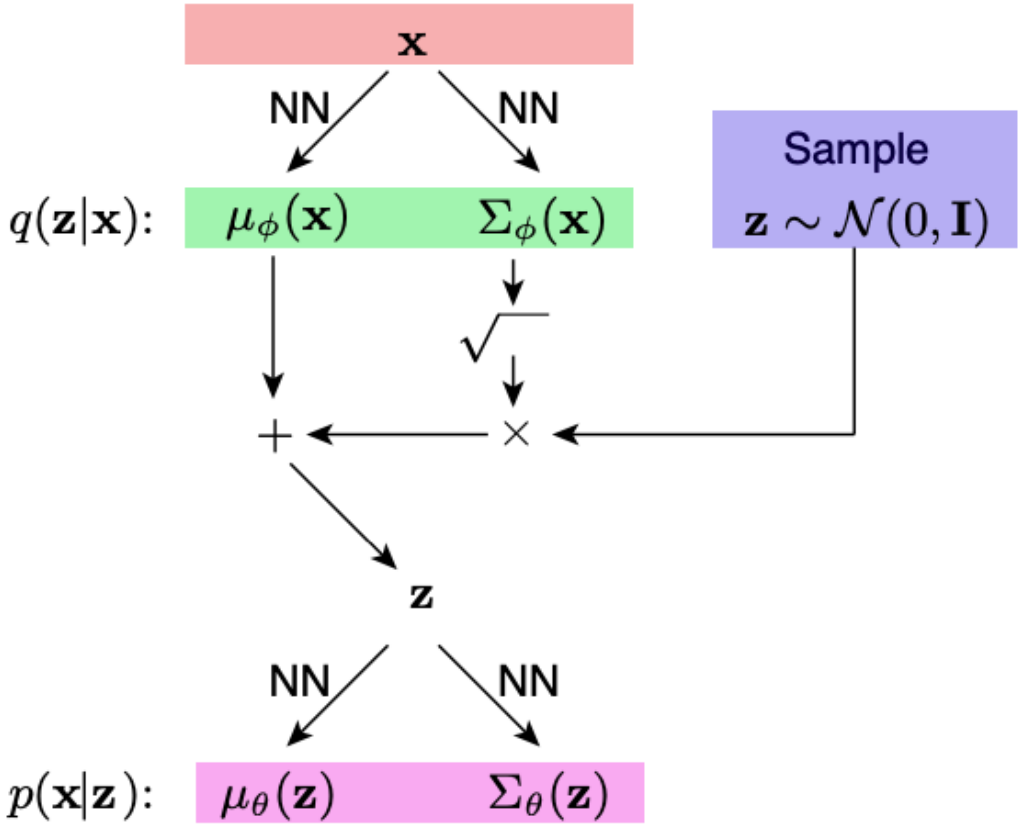
$$z \sim q(z|x)$$

- Can be done:

$$\varepsilon \sim N(0, I);$$

$$z = \mu_\phi(x) + \Sigma^{\frac{1}{2}}(x)\varepsilon.$$

- Backpropagation now does not pass through sampling.



# Final Per-event Objective and Results

- Write out

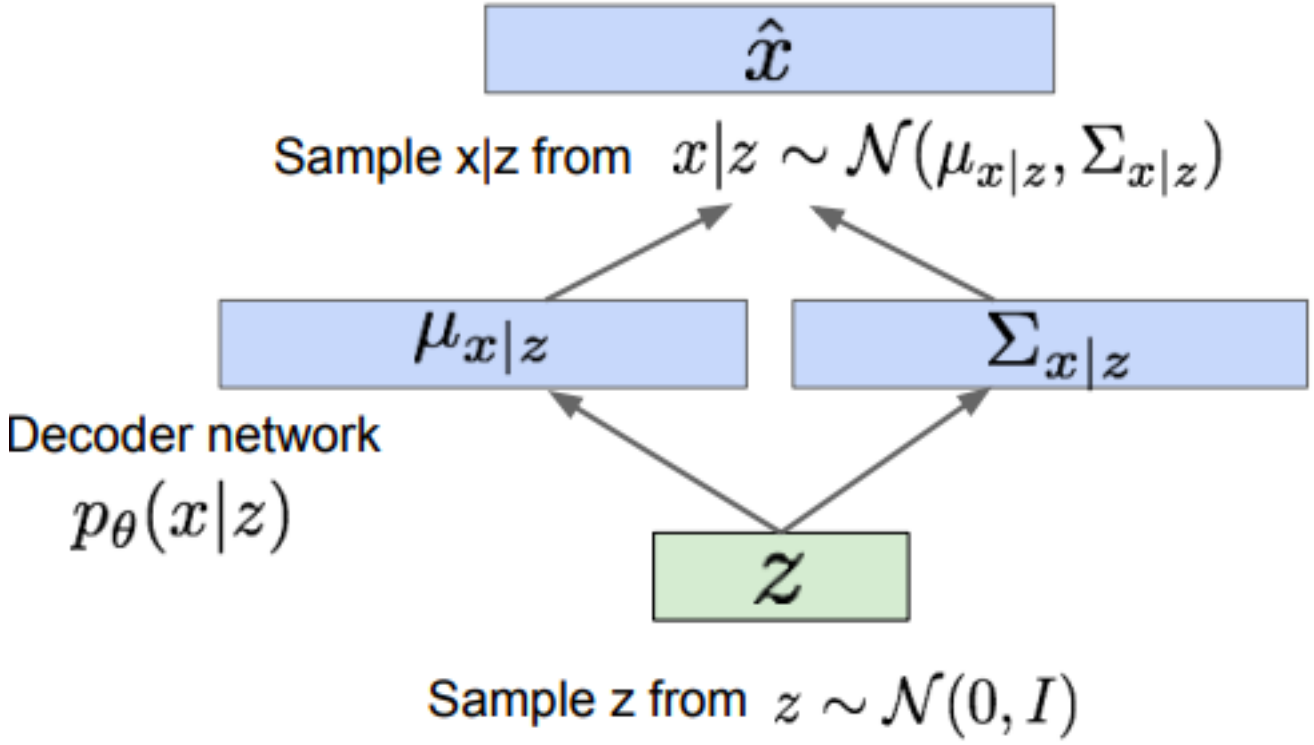
$$\mathcal{L}_{VAE}\,(x_i;\theta,\phi) = \mathbb{E}_{z\sim p(\varepsilon)}\log[p(x_i|z;\theta)] - KL[q(z|x_i,\phi),p(z)]$$

- We can even use a single noise sample:

$$\boxed{\mathcal{L}_{VAE}\,(x_i;\theta,\phi) = \log[p(x_i|z;\theta)] - KL[q(z|x_i,\phi),p(z)]}$$

- Note that the gradient estimates from per-event ELBO will be unbiased.

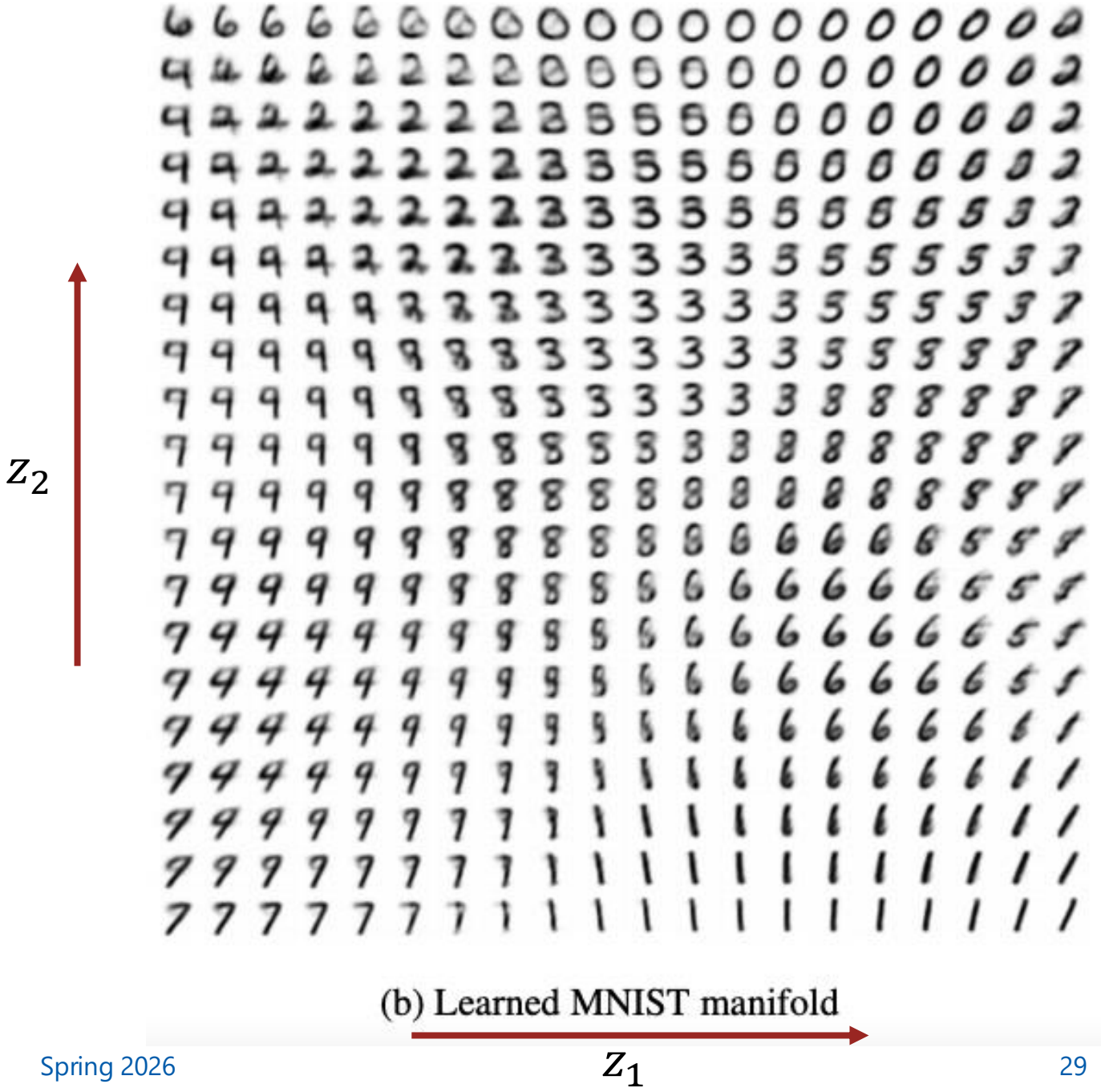
# Sampling




- We now can use the same idea like in GMM.
- Sampling from prior.

# Results

- The latent space is well organized.
- The results are good.




(b) Learned MNIST manifold

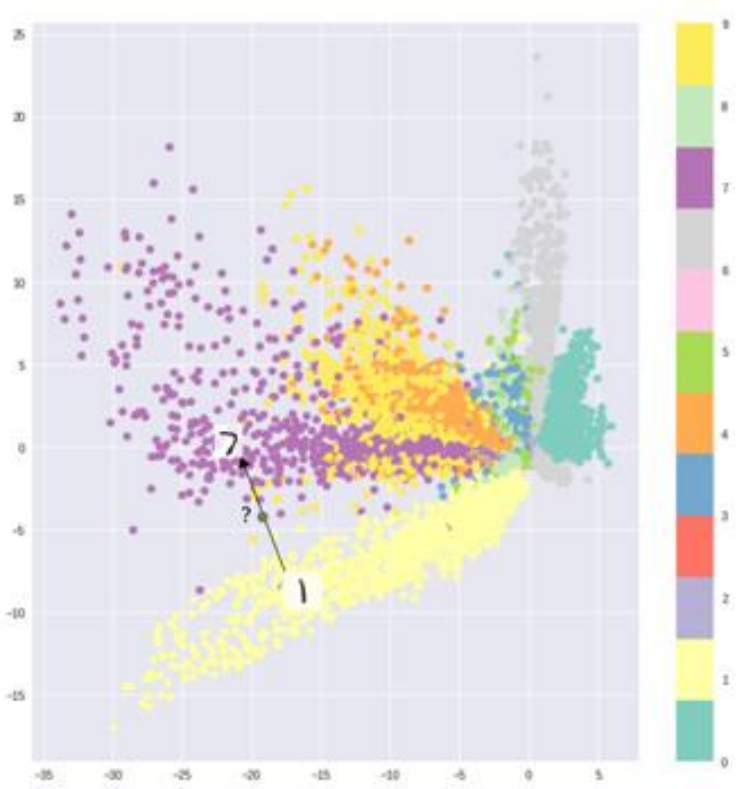
# Results

(a) 2-D latent space (b) 5-D latent space (c) 10-D latent space (d) 20-D latent space

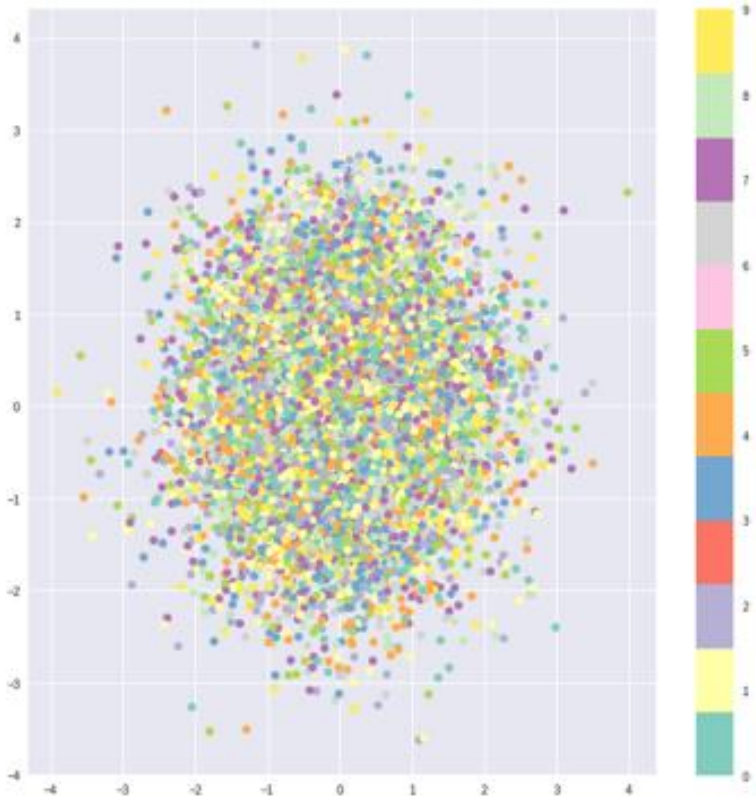
Figure 5: Random samples from learned generative models of MNIST for different dimensionalities of latent space.

# Results Discussed

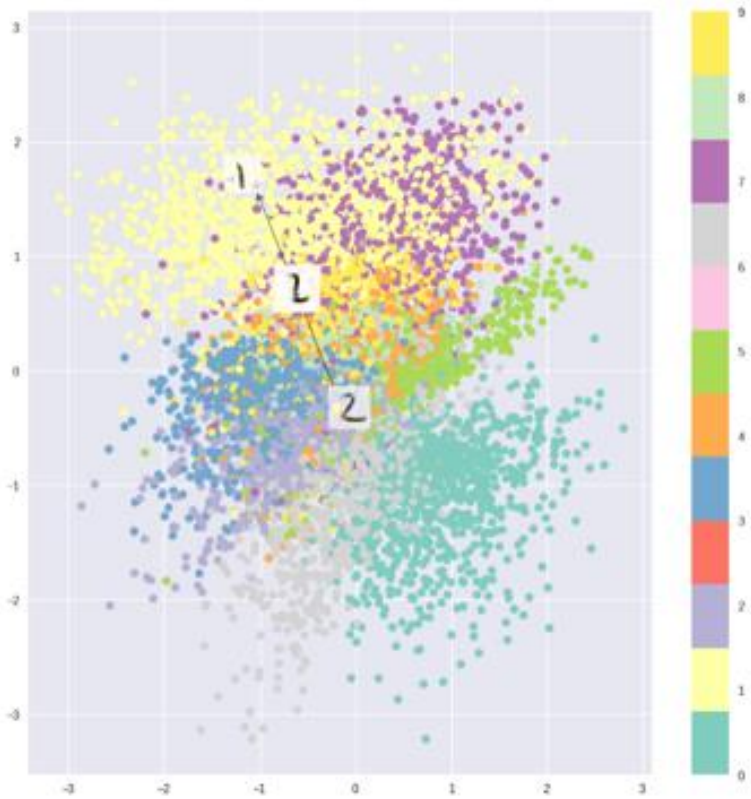
Only reconstruction loss




Only KL divergence



Combination




Shafkat's blog

# Amortized Inference

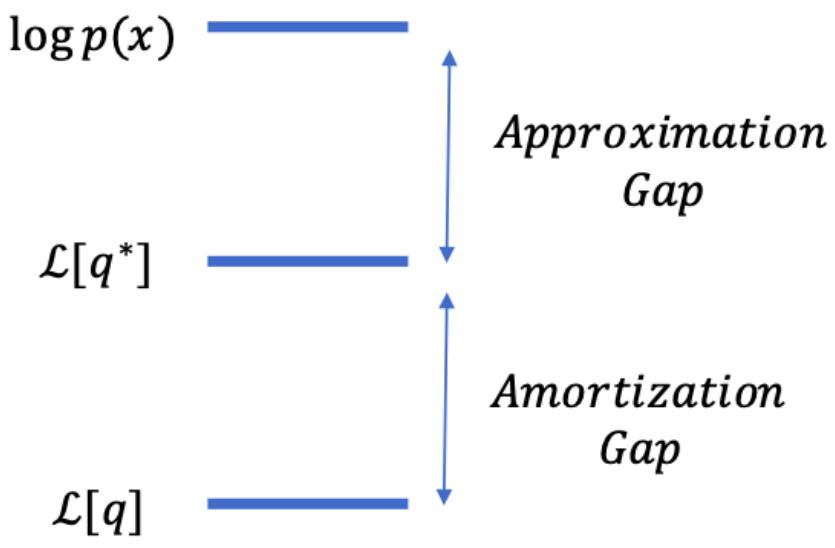



*Figure 1.* Gaps in Inference

- variational parameters $\phi_i$ should be calculated for every data point $x_i$;
- for big datasets this is unsustainable;
- forget about it and make a single model.

Cremer et al Inference Suboptimality in Variational Autoencoders, ICML 2018

# Images for Amortization Gap

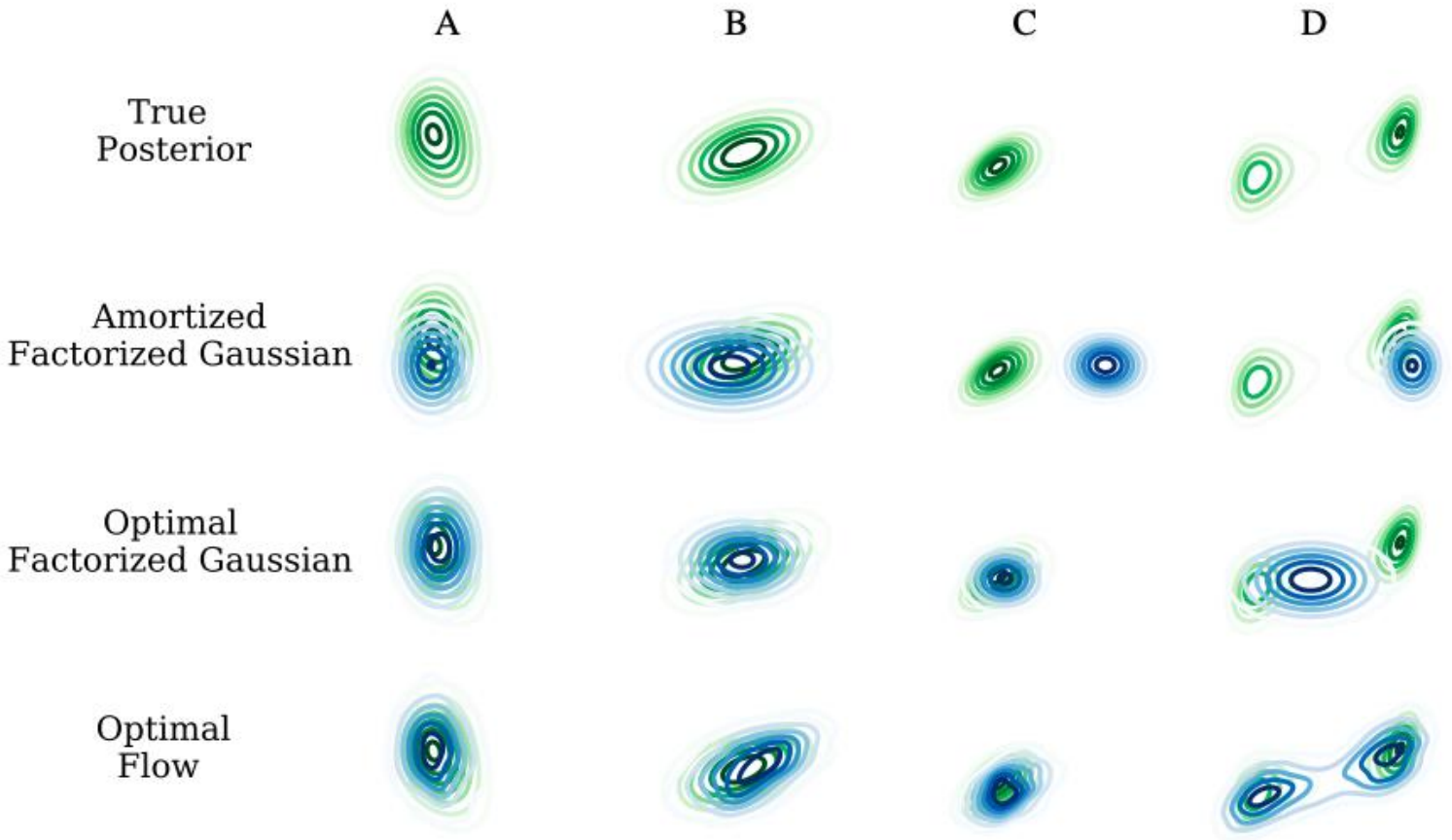



*Figure 2.* True Posterior and Approximate Distributions of a VAE with 2D latent space. The columns represent four different datapoints. The green distributions are the true posterior distributions, highlighting the mismatch with the blue approximations. Amortized: Variational parameters learned over the entire dataset. Optimal: Variational parameters optimized for each individual datapoint. Flow: Using a flexible approximate distribution.

# VAE: starting optimization problem

- Stochastic optimization can get stuck:
  - in the beginning, likelihood term $\log p(x|z)$ is relatively weak, such that an initially attractive state is where $q(z|x) \approx p(z)$.
- Several ideas possible:
  - optimization schedule - gradually increase weight of KL.
  - free bits - ensure that on average, a certain minimum number of bits of information are encoded per latent variable.

D. Kingma et al., An Introduction to Variational Autoencoders

# VAE: posterior collapse

Can happen to weak encoders or noisy data.

The sampling does not depend on the latent model.

An averaged figures are produced.

KL-divergence term annealing is proposed.

Fig: J. Chou Generated Loss and Augmented Training of MNIST VAE

# VAE: blurred problem

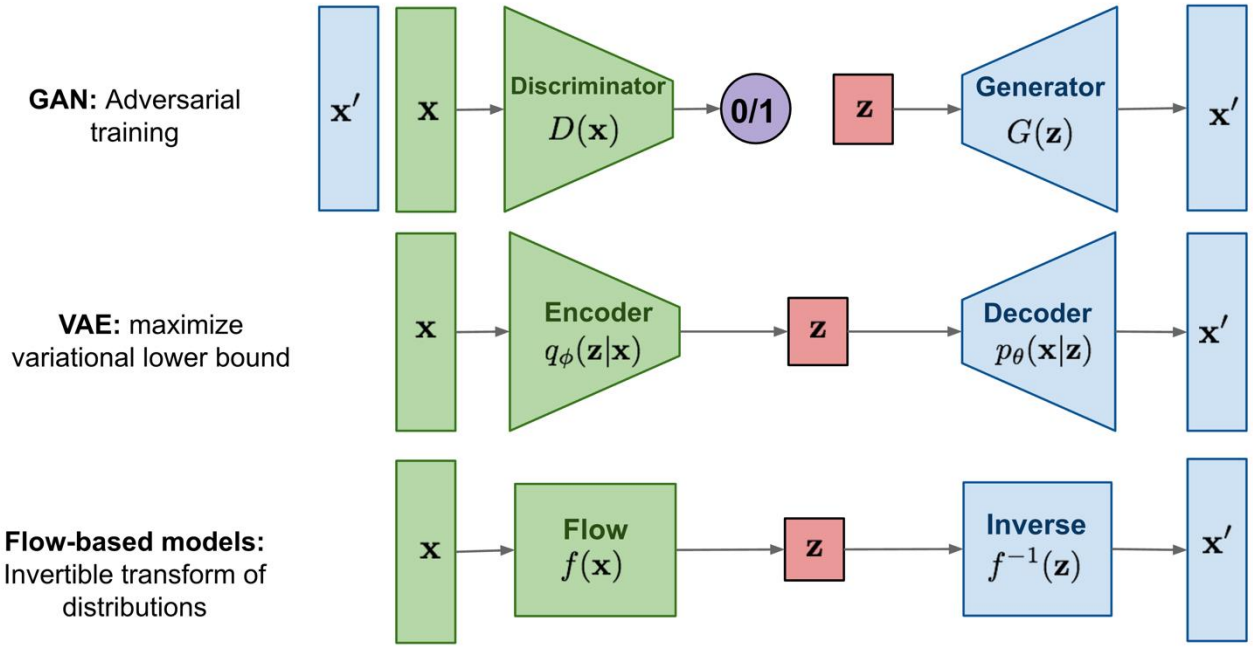
Due to minimization of KL
$KL(q(z;\phi)||p(z|x;\theta)) \to 0.$

Can be countered by choosing a sufficiently flexible inference model, and/or a sufficiently flexible generative model.

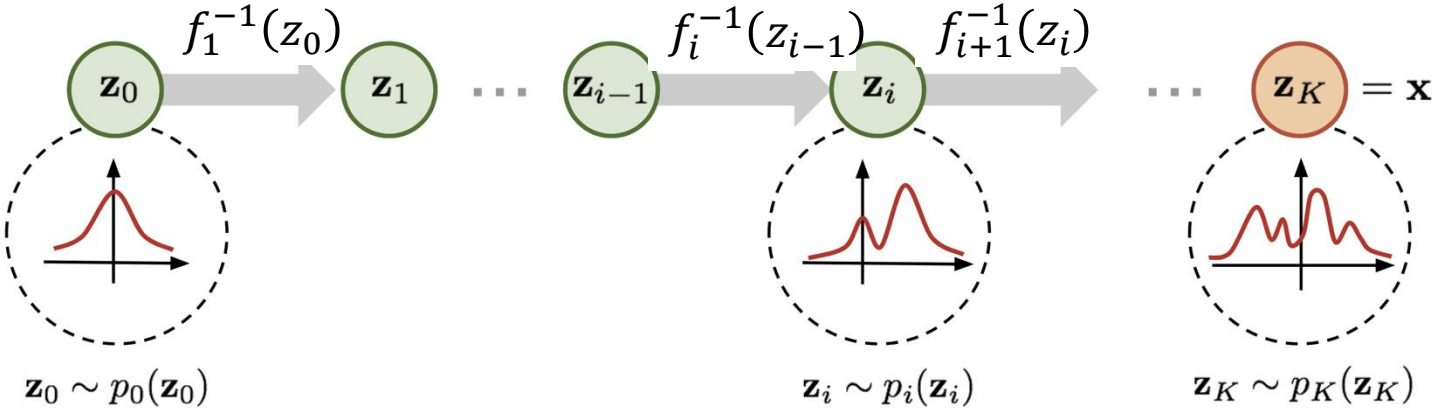
# Diffusion Motivation

# В предыдущих сериях

- Нестабильный процесс обучения GAN
- Размытые и нереалистичные результаты VAE
- GAN и VAE чрезмерно сложны (как минимум одна дополнительная сеть используется в процессе обучения)
- Нормализующие потоки ограничены из-за быстрого вычисления якобиана и обратимости



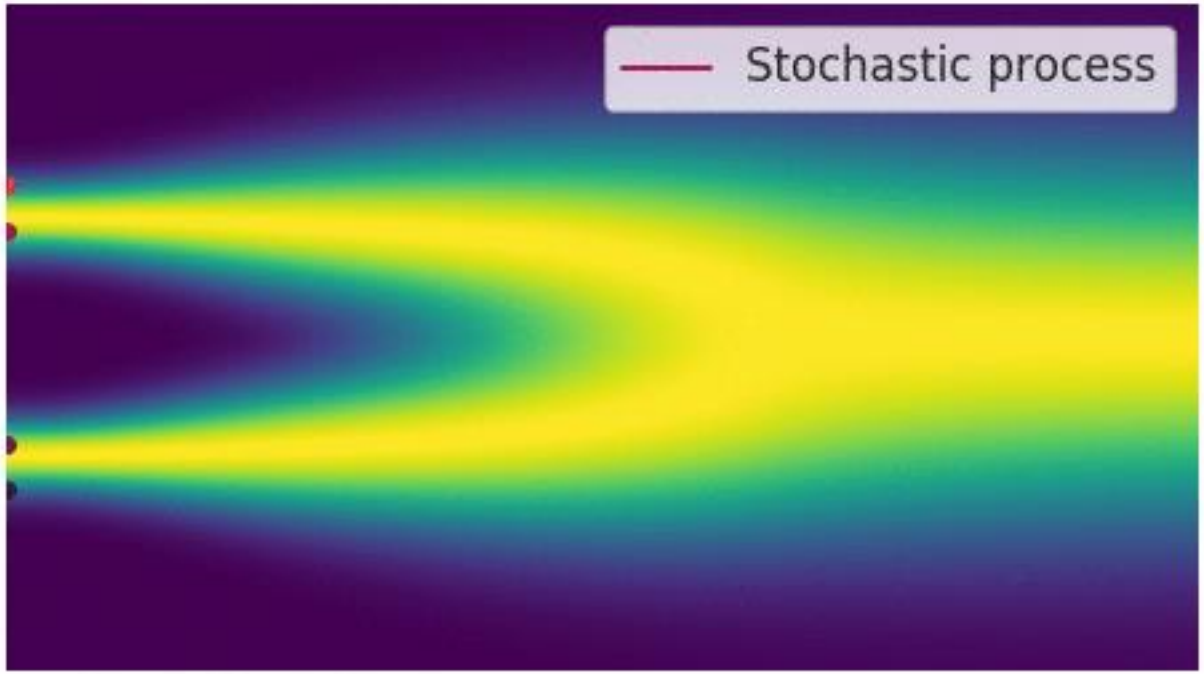
# Flow discussion




$$p_x(x) = \boldsymbol{p_{z_0}}\left(\boldsymbol{f_K}\left(\ldots\left(\boldsymbol{f_1}(x)\right)\right)\right)\left|\det\frac{\partial \boldsymbol{f_1}(z_1)}{\partial z_1}\right|\ldots\left|\det\frac{\partial \boldsymbol{f_K}(z_k)}{\partial z_k}\right|.$$

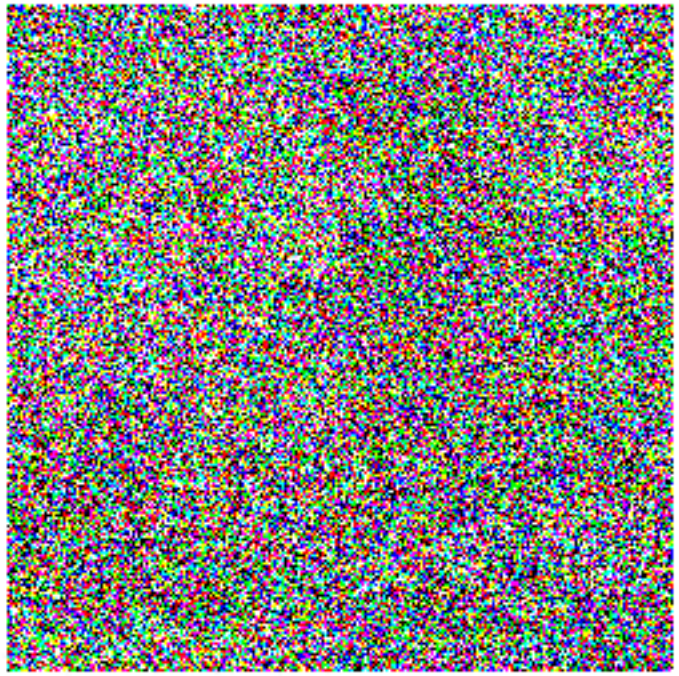
- It is possible to obtain $p_x(x)$ consecutively changing observables.
- The overall transformation is **invertible** if individual layers are invertible.
- Dimensions of each observable is the same.
- Fit is performed using ML estimate.
- **Need to calculate determinant.**

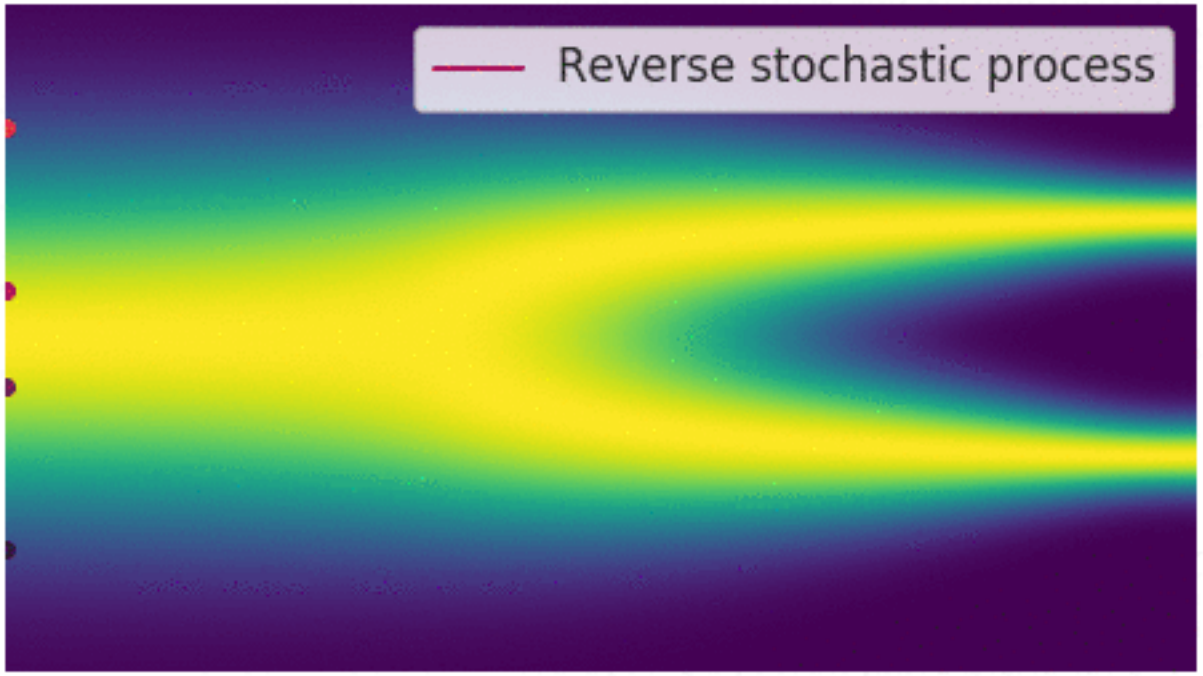
# Forward diffusion




https://yang-song.net/blog/2021/score/

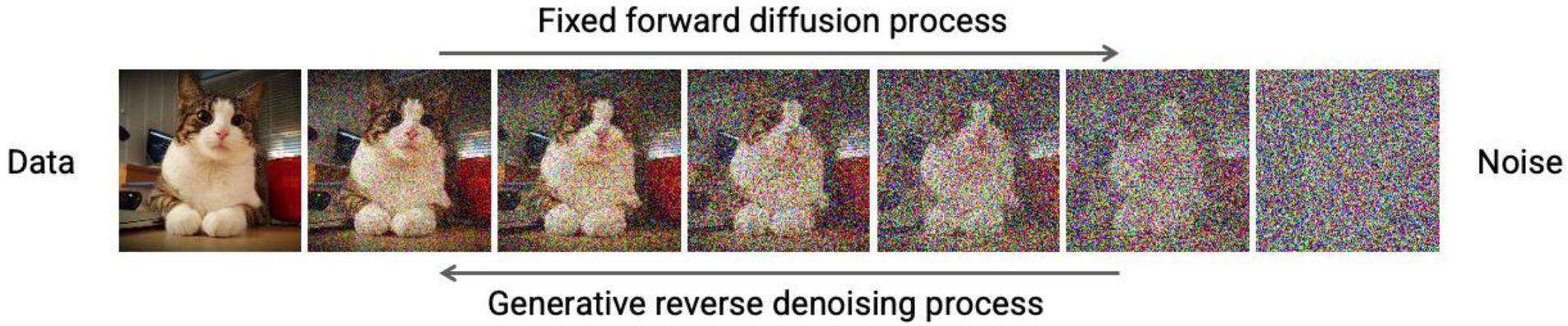
# Reverse diffusion






https://yang-song.net/blog/2021/score/

# Diffusion process

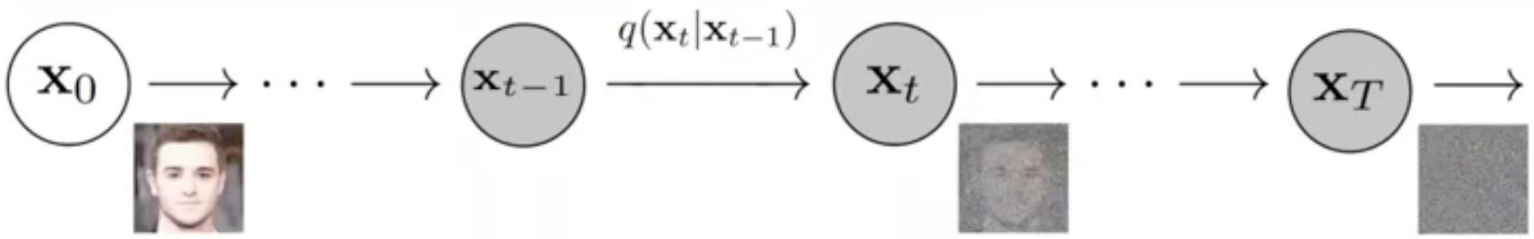



- Forget invertibility
- Fix forward process
- Learn approximate likelihood estimation

https://cvpr2022-tutorial-diffusion-models.github.io/

# Diffusion Models

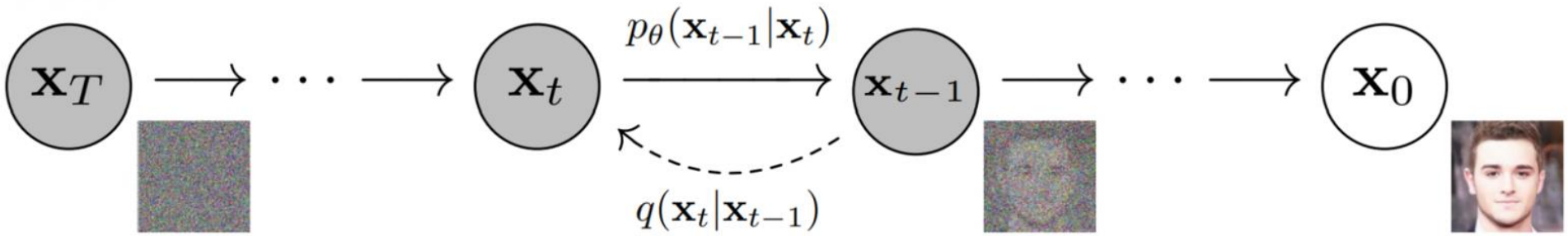
# Forward Diffusion




- $x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1-\alpha_t}\epsilon_t$ $\quad \epsilon_t \sim N(0, I), \quad \alpha_t = 1 - \beta_t$
- $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ $\quad \epsilon \sim N(0, I), \quad \bar{\alpha}_t = \prod_{i=1}^{t} \alpha_t$
- $x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right) + \tilde{\beta}_t z$ $\quad z \sim N(0, I), \quad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$

https://cvpr2022-tutorial-diffusion-models.github.io/

# Inverse Diffusion



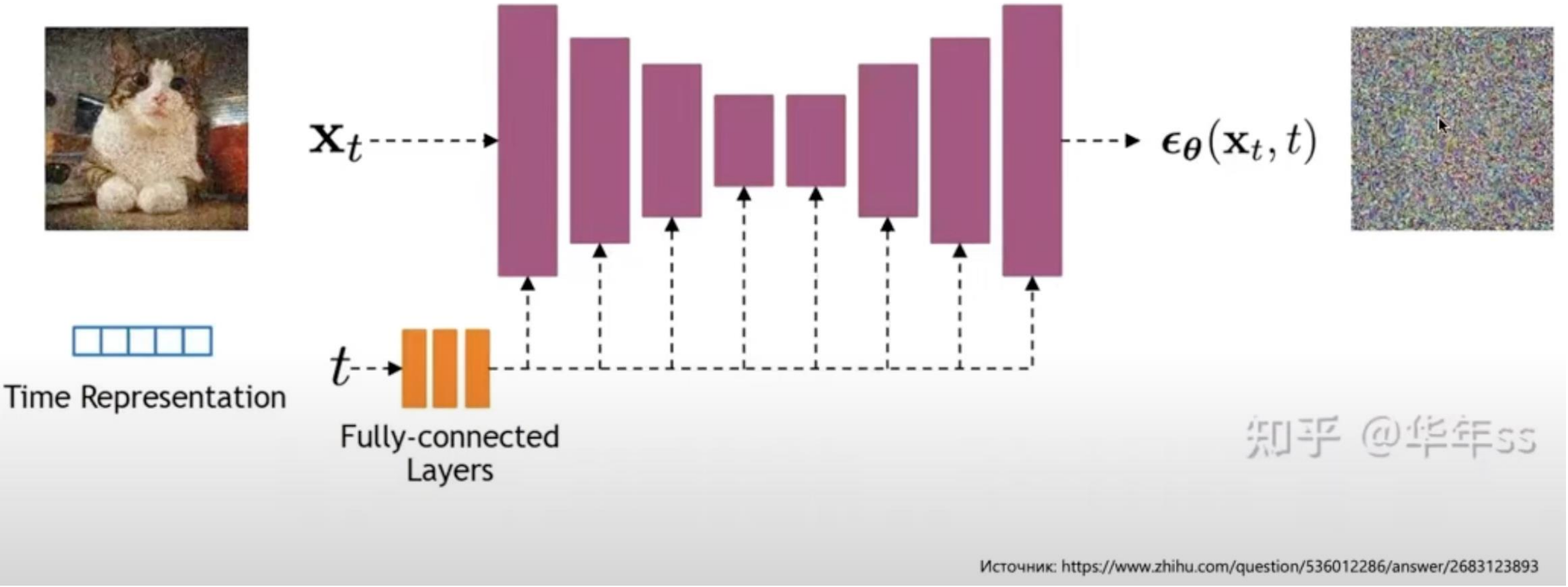

$$\hat{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right) + \sigma_t z \qquad z \sim N(0, I), \quad \sigma_t = const$$

Loss function:

$$L_t = \|x_{t-1} - \hat{x}_{t-1}\|_2^2 \propto \|\epsilon - \epsilon_\theta(x_t, t)\|_2^2 \to \min_\theta$$

https://cvpr2022-tutorial-diffusion-models.github.io/

# U-net architecture for diffusion



# Training/Sampling

**Algorithm 1** Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

**Algorithm 2** Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

https://arxiv.org/abs/2006.11239

# Problem!

**Problem**: How to train $p_\theta$? Absolutely the same as with VAE!

$$
\begin{aligned}
-\log p_\theta(\mathbf{x}_0) &\leq -\log p_\theta(\mathbf{x}_0) + D_{\mathrm{KL}}(q(\mathbf{x}_{1:T}|\mathbf{x}_0)\|p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0)) \\
&= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_{\mathbf{x}_{1:T}\sim q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})/p_\theta(\mathbf{x}_0)}\right] \\
&= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_q\left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} + \log p_\theta(\mathbf{x}_0)\right] \\
&= \mathbb{E}_q\left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})}\right] \\
\text{Let } L_{\mathrm{VLB}} &= \mathbb{E}_{q(\mathbf{x}_{0:T})}\left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})}\right] \geq -\mathbb{E}_{q(\mathbf{x}_0)} \log p_\theta(\mathbf{x}_0)
\end{aligned}
$$

See Seminar for detail

# Results

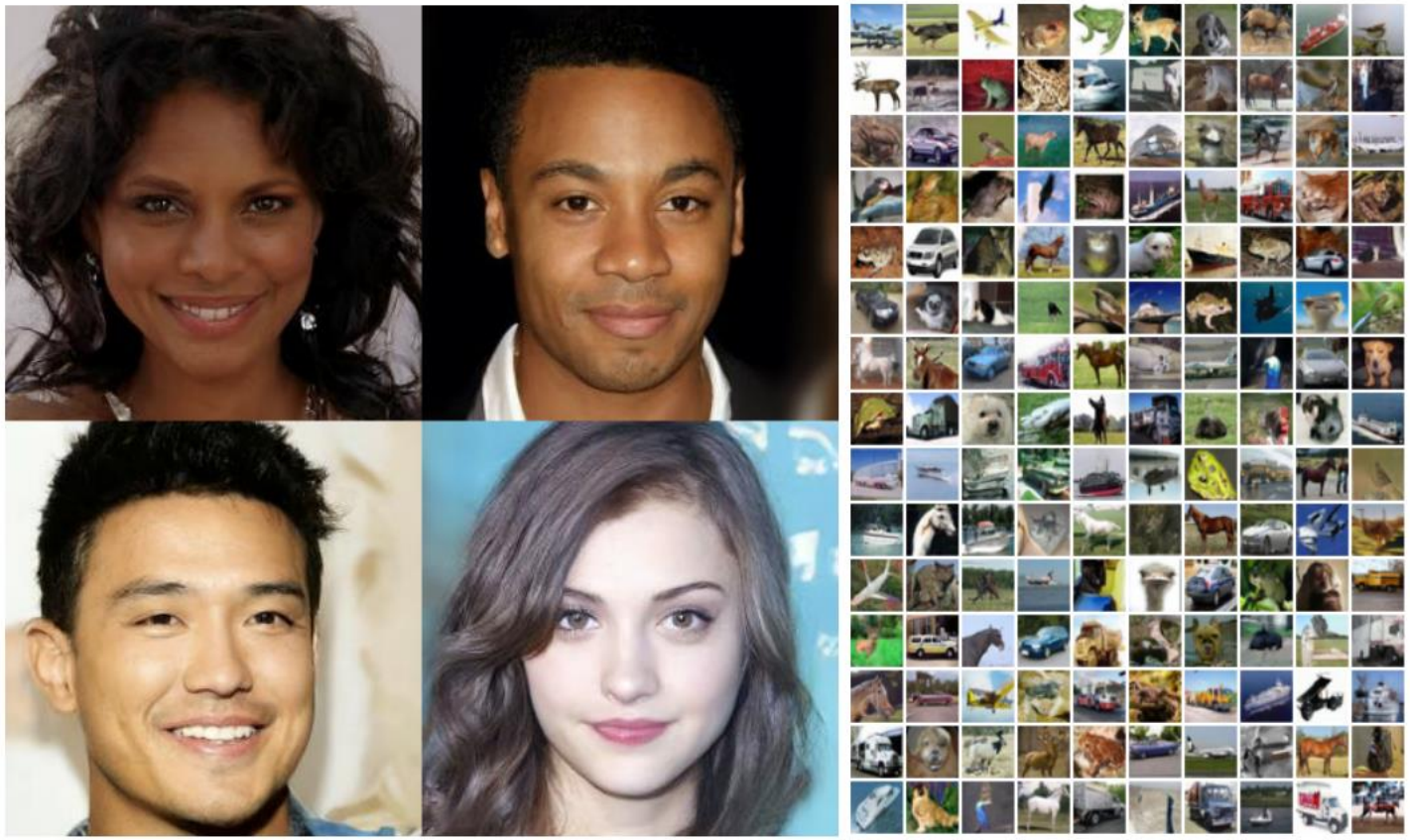


Figure 1: Generated samples on CelebA-HQ 256 × 256 (left) and unconditional CIFAR10 (right)

https://arxiv.org/abs/2006.11239

# Conclusions

- Very Good Quality
- Closely related to Stochastic Differential Equations
- No Inner representation
- Should keep the same dimensions at each step